Prometheus Monitoring Architektur

Monitoring Ziele: Alerting (Alarm bei Störung), Debugging (Ursachen analyse), Trending (Kapazitätsplanung).

Abgrenzung verwandter Techniken

- Profiling: kurze hochdetaillierte erfassung aller Daten z.b Wireshark
- Tracing: Verfolgung einzelner Requests über verteilte Systeme via trace ID
- Logging: Textueller Mitschrift von Ereignissen mit Kontext (z.b Minecraft Server)

Prometheus Architektur

Pull-Mechanismus (Scraping via HTTP im 10-60s Intervall von Exportern/Endpoints.) Lokale TSDB-Datenbank
AlertManagging Per E-Mail/Webhook.

Prometheus konfiguration (/etc/prometheus/prometheus.yml):

```
global: scrape_interval: 15s
evaluation_interval: 15s
scrape_configs:
  - job_name: 'node_exporter'
    static_configs:
    - targets: ['localhost:9100', '159.100.256.96:9100']
```

promtool check config /etc/prometheus/prometheus.yml
sudo systemctl reload prometheus.service

→ Node Exporter sammelt System infos auf dem port 9100.

Prom QL Query Language

Die Vier Metriken:

-> Gauge Pegelstand: Momentanwert der steigt oder fällt
Suffixe: _bytes, _seconds
Bsp: node_filesystem_size_bytes.

-> Counter Zähler: Kommulierter Wert der immer Ansteigt.
Raten bildung mit rate()
Suffix: _total
Bsp: node_network_recieve_bytes_total

-> Summary Statistiken: Paarweise Metriken mit _sum & _count
Beispiel: prometheus_http_response_size_bytes_sum

-> Histogramm: Gruppiert Messwerte in Buckets. Suffix: _buchet
Bsp: prometheus_http_request_duration_seconds_buchet

Target Labels vs. Instrumentationslabels

Target Labels: Identifizieren das überwachte zielsystem (job, instance)
Instrumentations-Labels: Metrik spezifischer Kontext (device, fstype, mountpoint)
Prom QL behandelt beide gleich.

Filtering & Regex Matching:

```
node_filesystem_size_bytes{fstype="ext4"}
node_cpu_seconds_total{mode=~"user|system",
instance="localhost:9100"}[30m]
```

Zeitfenster & Ratenbildung:

```
rate(node_network_recieve_bytes_total[5m])
last_over_time(promhttp_metric_handler_requests_total[1h])
```

Summary Durchschnittsberechnung

```
sum(rate(prometheus_http_response_size_bytes_sum[5m]))
sum(rate(prometheus_http_response_size_bytes_count[5m]))
```

Histogram Quantile & Median

```
histogram_quantile(0.5,
rate(http_request_duration_seconds_bucket[5m])) -> Median (50% schneller/langsamer)
histogram_quantile(0.9,
rate(http_request_duration_seconds_bucket[5m])) -> 90%-Quantil
```

Aggregationen mit by/without:

```
sum without (job, instance) (node_filesystem_size_bytes)   ->Entfernt Target Labels mit "by",
sum by (device,fstype,mountpoint) (node_filesystem_size_bytes)-äquivalent
```

Arithmethik Logik & Zeit

```
Bytes in GB: node_filesystem_size_bytes / (1024^3)
Filter >1GB: node_filesystem_size_bytes/(1024^3) >1
```

Zeitrechnung: time() - 24*60*60 (gestriger zeitstempel)

Frage 6: Valkey Hashes Auslesen
F: Befehl zum Auslesen aller Key-Value-Paare eines Hashes?
L: HGETALL key

Frage 7: Valkey Atomare Listen-Operation
F: Welcher Befehl verschiebt atomar ein Element von Liste A nach B?
L: RPOPLPUSH source_list dest_list

Frage 8: s3fs Dateirechte
F: Welches Dateirecht verlangt ~/.passwd-s3fs zwingend?
L: Exakt chmod 600 (nur Owner darf lesen/schreiben).

Frage 9: Prometheus Architektur
F: Arbeitet Prometheus mit Push oder Pull?
L: Primär mit Pull (Scraping via HTTP im 15-60s Intervall von Exportern).

Frage 10: Node Exporter Rolle
F: Welche Aufgabe hat der Node Exporter?
L: Stellt OS- & Hardware-Metriken (CPU, RAM, Disk, Net) auf Port 9100 bereit.

Frage 11: PromQL histogram_quantile
F: Bedeutung von histogram_quantile(0.5, ...) = 4096?
L: 4096 ist der Median (50%-Quantil). 50% aller Allokationen waren ≤ 4096 Bytes.

Frage 12: PromQL Label Aggregation
F: Was bewirkt sum without(job, instance)(...)?
L: Es entfernt die Server-Labels (Target-Unterscheidung) aus dem Ergebnis.

Frage 13: PromQL Ratenberechnung
F: Welche Funktion berechnet den pro Sekunde gemittelten Zuwachs eines Counters?
L: rate(metric[5m])

Frage 14: PromQL Byte-Umrechnung
F: Wie rechnet man Bytes in PromQL in GB um?
L: metric / (1024^3)